

Project: Movie Service Mashup

September 17, 2026

1 The Model

Walter White, a movie lover, has created a database (model) in which he stores the following information about the films he has watched over the years: • **Title:** the title of the movie, • **Year:** the year the movie was released, • **Viewing date:** the date on which Walter watched the film, • **Score (0–10):** the score Walter gave to the film. The model follows the diagram in Figure 1.

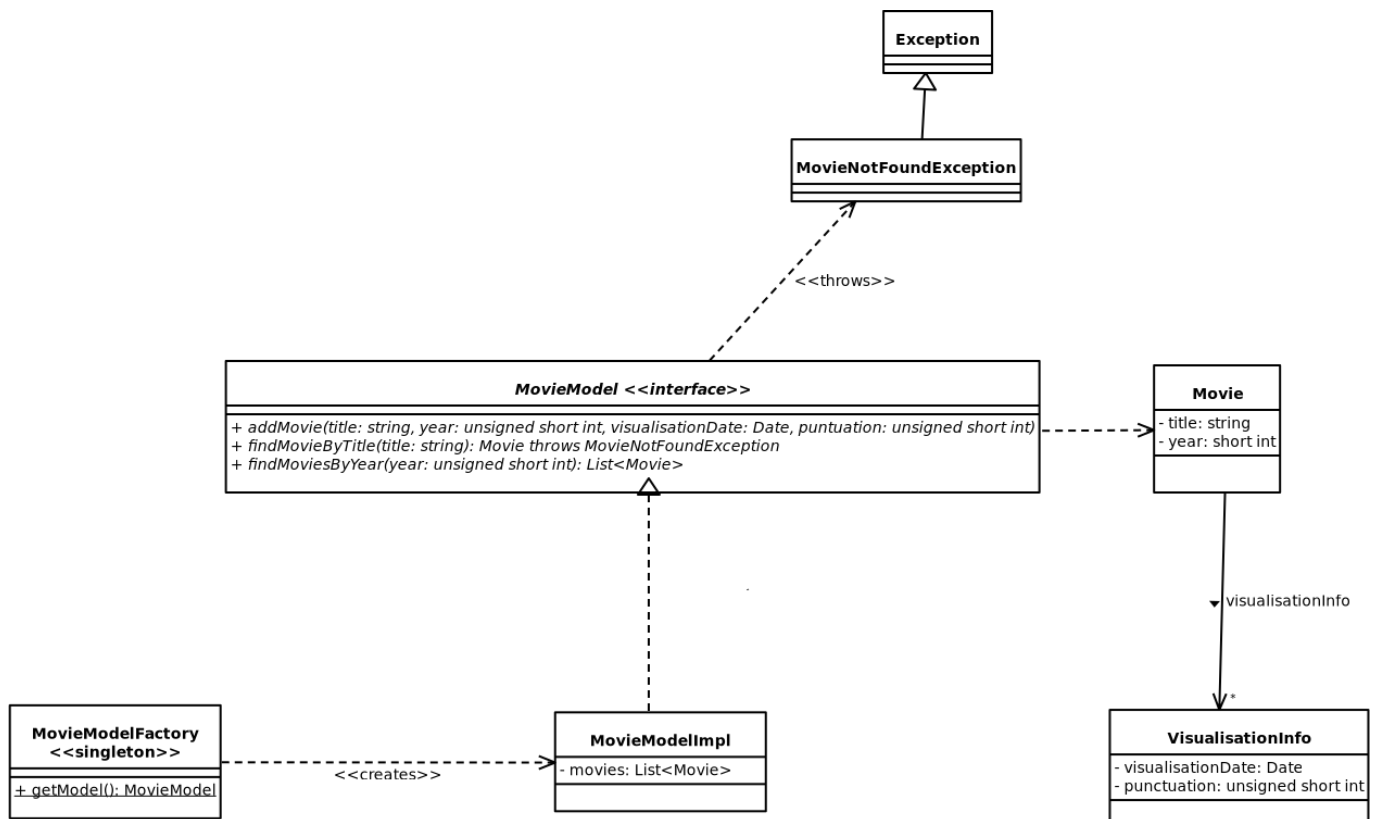


Figure 1: Class diagram describing the model.

Exercise 1

In this subproject, implement the model and the operations described in the class diagram of Figure 1. Take into account the following points:

- **addMovie:** this method adds a movie to the model. If the movie already exists, its content is updated¹.
- **findMovieByTitle:** this method returns the movie whose title is passed as a parameter. If the movie does not exist, it throws a `MovieNotFoundException`.
- **findMoviesByYear:** this method returns all movies seen during the year passed as an input parameter.

The static method `MovieModelFactory.getModel()` returns an instance that implements `MovieService`. Note that this class is a **singleton**. The list `movie` within the class `MovieModelImpl` can be populated when the class is loaded².

¹We assume that the title plays the role of identifier.

²This can be done by means of a `static` block within `MovieModelImpl`.

2 Thrift Web Service

We want to export the operations provided in the model described in Figure 1. The class diagram of this Thrift web service is shown in Figure 2.

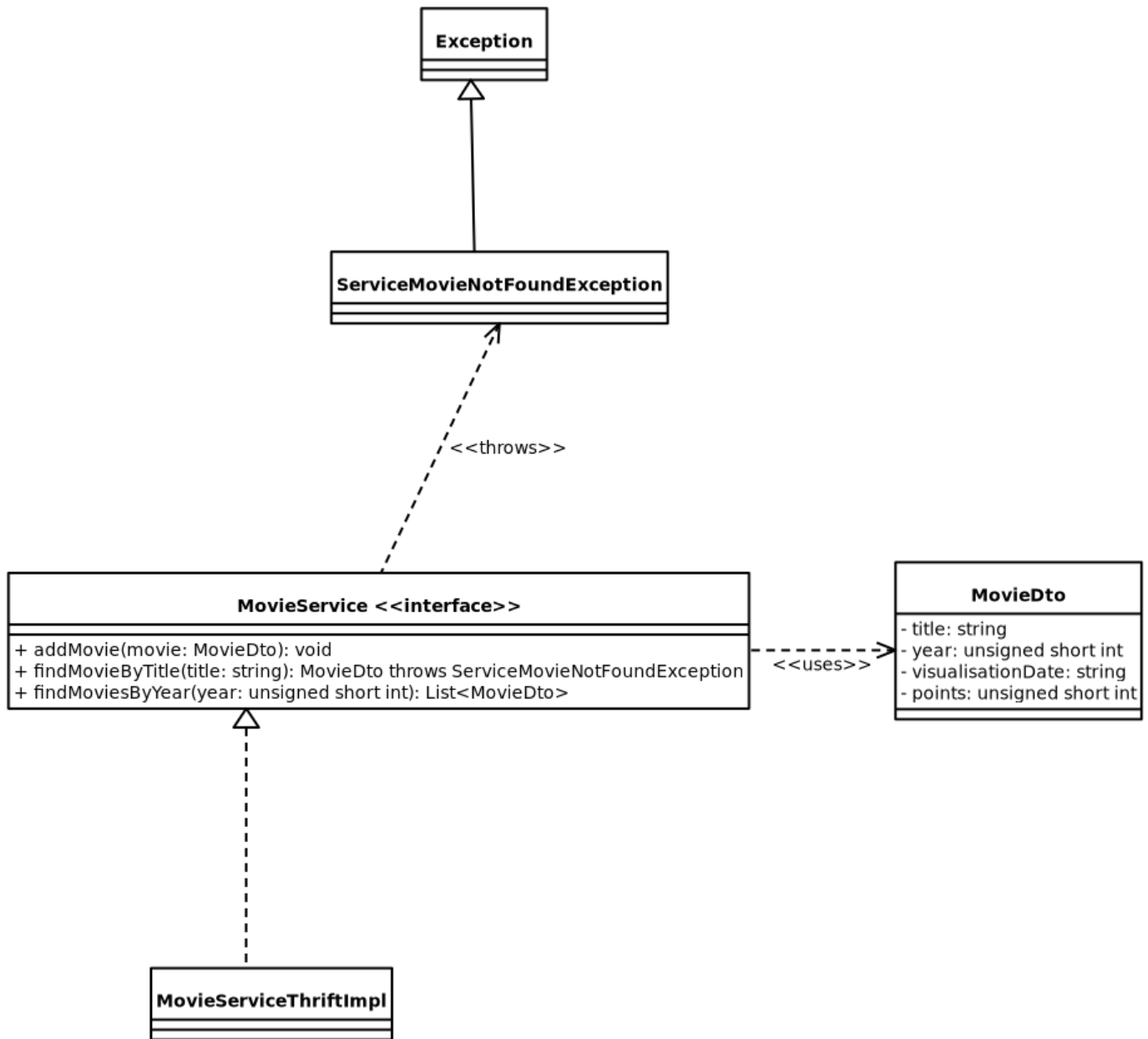


Figure 2: Class diagram representing the structure of the Thrift web service.

Note the difference between the classes **MovieDto** and **Movie**, which makes them incompatible.

Exercise 2

In the subproject `thrift`, develop a Thrift web service that implements the class diagram in Figure 2. Take into account the following items:

- The operation `addMovie` takes as input an object **MovieDto** and creates the corresponding entry in the model.
- The operation `findMovieByTitle` retrieves a movie with a given title from the model. If the movie does not exist, a **ServiceMovieNotFoundException** is thrown.
- The operation `findMoviesByYear` provides the list of movies (in terms of a list of objects **MovieDto**) that were seen during a given year.

3 TheMovieDB REST API

TMDB (The Movie Database) provides a free API offering movie, TV show, and actor information, such as searching for films and actors/actresses, as well as images and posters of films. It is also free for non-commercial use. Its use

is particularly convenient because films and actors can be searched by title, after which detailed information about a specific film can be retrieved via its `id`. For instance, the following [query](#) provides information about the film *The Matrix* (among others), whose `id` is 603. To get the details of a given film, it is necessary to access it via its `id`, which returns the following JSON code.

```
{
  "adult": false,
  "backdrop_path": "/lrtSb1skJayPydZk0OSMAKjBOVe.jpg",
  "budget": 63000000,
  "genres": [ { "id": 28, "name": "Action"}, {"id": 878, "name": "Science Fiction"} ],
  "id": 603,
  "overview": "Set in the 22nd century...",
  "popularity": 48.6817,
  "poster_path": "/a0IuZAJPaRIE6CMzbazvcHuHXDc.jpg",
  "title": "The Matrix",
  ....
}
```

For a given film, say *The Matrix*, its associated credits can be retrieved via a [GET](#) request:

```
{"id": 603,
  "cast": [{
    "gender": 2, "id": 6384, "name": "Keanu Reeves", "original_name": "Keanu Reeves",
    "profile_path": "/8RZL0yYGsoRe9p44q3xin9QkMHv.jpg", "character": "Neo",
  },{
    "gender": 2, "id": 2975, "name": "Laurence Fishburne", "original_name": "Laurence Fishburne",
    "profile_path": "/2GbXERENPpl5MmlqOLlPVaVtifD.jpg", "character": "Morpheus",
  },{
    "gender": 1, "id": 530, "name": "Carrie-Anne Moss", "original_name": "Carrie-Anne Moss",
    "profile_path": "/xD4jTA3KmVp5Rq3aHcymL9DUGjD.jpg", "character": "Trinity",
  }, .....]
}
```

For each character, say [Keanu Reeves](#), the biographical information can be retrieved via a simple [GET query](#)

```
{
  "birthday": "1964-09-02", "deathday": null, "id": 6384, "name": "Keanu Reeves",
  "place_of_birth": "Beirut, Lebanon", "profile_path": "/8RZL0yYGsoRe9p44q3xin9QkMHv.jpg"
}
```

You will combine the three different queries in order to retrieve custom information about a film.

Exercise 3

Create a client for the RESTful [TMDB API](#) that follows, as much as possible, the class diagram of [Figure 3](#). The operation `findMovieInformation(title: string): MovieInfoDto` returns the information associated with a film (object `MovieInfoDto`) whose title is passed as an input parameter. The resulting `MovieInfoDto` object contains the `title`, a list of `genres`, the full URL associated with its `poster` picture, and the `release year`. In addition, it will provide a list of `characters`. For each character, the application will return both the `character` and `actor` names, the complete URL to the actor's `picture`, the `birthday` and `deathday`, and the `place of birth`. The factory that creates a client behaves, independently of the concrete implementation, as a `singleton`.

4 VirtualMovieInformationService

In this section, you will develop a REST web service that combines the client developed in [Section 3](#) with the Thrift web service developed in [Section 2](#). This new web service is called `VirtualMovieInformationService`. This service will be developed using the [SpringBoot](#) framework.

Exercise 4

Develop the REST web service `VirtualMoviesInformationService` that publishes the following operation:

```
findMovieInformation(title:string): VirtualServiceMovieDT0.
```

This operation will use the client developed in [Section 3](#) and the Thrift client developed in [Section 2](#) to retrieve the existing information about a film whose title is passed as an input parameter. The operation `findMovieInformation`

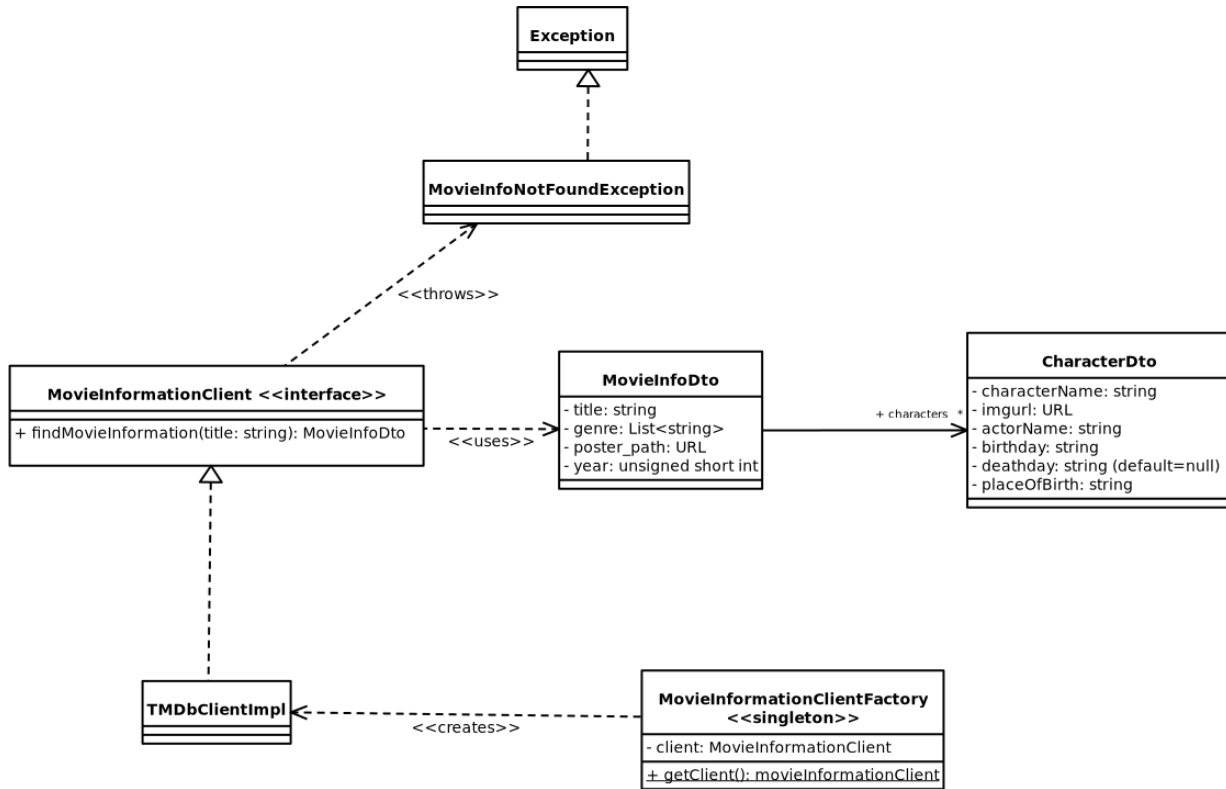


Figure 3: Class diagram that represents the TMDb client.

combines the information retrieved from the TMDb client with the information retrieved from the Thrift client³ developed in Section 2 into a single DTO object called `VirtualServiceMovieDTO`. The definition of the `VirtualServiceMovieDTO` and the structure of the service are not provided; they are left to the developers. The use of design patterns such as factories or facades, as well as an appropriate use of inheritance, will be taken into account. Finally, error codes must be mapped into HTTP response codes in a “logical” way.

5 The client

The last exercise consists of programming a (command-line) client that allows the user to invoke the different functionalities of the system.

Exercise 5

Develop a command-line client that allows the user to invoke the following methods:

1. `findMovieInformation(title:string)` from `VirtualMoviesInformationService` described in Section 4.
2. `addMovie(MovieDto)` from the Thrift web service of Section 2.

For each operation, the client will print the result obtained. The design decisions are left to the developers, but the use of design patterns and the corresponding DTOs is recommended in order to decouple the implementation of this client from other web services’ clients.

The final structure of this project is described in Figure 4.

6 Evaluation

6.1 Groups

The project will be carried out in groups of 3 people.

6.2 Coding Norms

In order to write high-quality, easy-to-read code, a common coding standard is used, which defines rules for naming classes, attributes and methods, identification rules, etc. This allows a development team to maintain a consistent appearance for the code regardless of who wrote it, which makes maintenance easier.

³Do not duplicate the information.

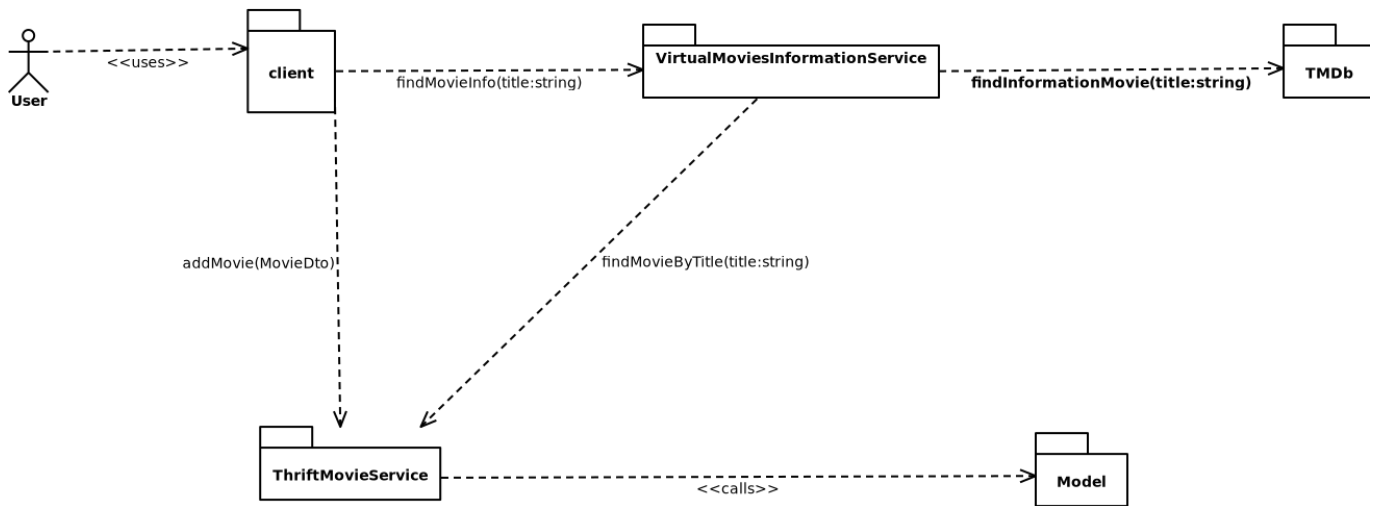


Figure 4: General Structure of the project

6.3 Submission of the Final Version

It will be mandatory to use the **git** version control system, hosted on GitHub or GitLab. Each group will provide a link to a [tag or release](#) corresponding to the final version of the project that will be used for grading. **The code will be submitted via GitHub instead of Moodle.** In addition, it will not be necessary to document the classes (for example, with JavaDoc).

The project will be developed in Java and should compile using the **gradle** command. In addition, the project should follow a multi-project approach (see the attached template).

6.4 Format of the Report

As a general rule, diagrams should use **UML** notation. Diagrams should be of high quality (not reverse engineered - design precedes implementation!) and should be briefly but clearly explained. The quality of the report will be essential for the evaluation of the project.

The report will contain the following parts:

Design

1. **Global Architecture:** briefly explain the modules that have been designed and implemented. Two "high-level" UML diagrams will be used to support the explanation.
 - A class diagram illustrating the abstractions (interfaces, *factories*, interface implementation classes, etc.) of the various modules involved in implementing the **VirtualMoviesInformationService**. This diagram should not show the names of the operations and attributes in the interfaces and classes. Its purpose is to illustrate the dependencies between the main abstractions.
 - A class diagram describing the main components of the client.
 - A sequence diagram illustrating the processing of an invocation of the **findMovieInformation** operation of the **VirtualMoviesInformationService**.
2. **i-th service/module:** For each service/module, it is necessary to write a section explaining its design. Each explanation should include:
 - The overall structure of the module package. It is not necessary to use UML notation. Nor is it necessary to show all the packages, just the top-level packages that reflect the architecture of the module and its main sub-packages. For each package, a brief explanation of what it contains should be given.
 - One or more diagrams illustrating the architecture of the module. In particular, the diagrams should specify in detail the operations of the interfaces and classes for **transfer objects**. For the rest of the abstractions (concrete classes), it will only be necessary to specify the operations and attributes that are considered relevant to understanding the explanation of the module (for example, an attribute that is important for understanding the explanation of an interface). In other words, UML diagrams should not bring together all the abstractions implemented, but only those that are relevant and, for each of them, show only what is necessary to understand the design of the module.

Compiling and installing the application It should be clearly explained how to compile and install the project, assuming a correctly configured environment (e.g. application server installed and configured, software packages installed in the same directories as in the lab sessions, etc.). The installation instructions should be as simple as possible. In particular, they should specify any particular aspects of the configuration that need to be highlighted⁴.

Known problems A list of known bugs/errors in the code.

Evaluation The evaluation of the project will take into account: 1) The code 2) The quality of the design 3) The quality of the report. A copied project means failure for both the group that authorized the copying and the group that copied; no distinction will be made.

Deadline 18/12/2026

⁴When grading the final version of each application, the installation instructions will be followed.